

Calculadora de Balanço de Massa e Energia

Versão: 2.0

Stack Principal: TypeScript (React) + Python (FastAPI) + Google Cloud Platform (GCP)

1. Visão Geral da Arquitetura e Filosofia do Sistema

O sistema foi concebido sob um modelo de **Arquitetura Serverless** baseada em containers, operando com microserviços fortemente desacoplados via REST API.

A filosofia central deste projeto é o isolamento de responsabilidades: o *Frontend* atua exclusivamente como uma camada de apresentação e coleta de dados de alta performance, enquanto o *Backend* encapsula 100% da inteligência físico-química e termodinâmica.

O fluxo operacional primário consiste no usuário interagindo com uma UI em React dividida em abas setoriais. Em vez de calcular os resultados em tempo real no navegador (o que travaria a *main thread* do JavaScript devido ao volume de dados), a interface acumula as alterações de estado. Ao clicar no botão "Calcular", um *payload* massivo contendo o estado de todos os 1108 campos é empacotado em JSON e enviado ao backend Python.

No backend, o sistema não resolve as equações de forma linear. Ele constrói dinamicamente um **Grafo Acíclico Dirigido (DAG)** para mapear a ordem exata de resolução das fórmulas (quem depende de quem) e então processa a recursividade termodinâmica até retornar o estado balanceado para a interface.

2. Pilha Tecnológica (Tech Stack) e Rationale de Seleção

A seleção tecnológica foi guiada pela necessidade de precisão matemática extrema, velocidade de processamento e manutenibilidade a longo prazo.

Camada	Tecnologia	Propósito / Rationale
Frontend	TypeScript / React (v18+)	Gestão de estado atômico e tipagem forte. O ecossistema React, aliado a bibliotecas de memoização, é a única forma segura de renderizar mais de mil inputs simultâneos sem gargalos de <i>repaint</i> no DOM.

Backend	Python 3.11+ / FastAPI	Alta performance matemática. O FastAPI oferece validação automática via Pydantic e execução assíncrona (ASGI). O Python foi escolhido pelo seu suporte nativo à biblioteca <code>decimal</code> , <code>Pandas</code> e robustez em cálculos científicos.
Banco de Dados	PostgreSQL (Cloud SQL)	Banco relacional escolhido pela garantia de ACID (Atomicidade, Consistência, Isolamento e Durabilidade), utilizando colunas <code>JSONB</code> para armazenar as 1108 variáveis sem a necessidade de criar um schema de banco de dados com 1108 colunas.
Hospedagem	Google Cloud Run	Deploy de containers <i>stateless</i> . Reduz custos operacionais para zero quando a calculadora não está em uso (escala a zero) e escala infinitamente durante picos de planejamento de safra. O <i>timeout</i> de 300s protege algoritmos longos como o <i>Goal Seek</i> .
Segurança	Google SSO (OAuth2)	Login corporativo restrito de forma nativa. O Cloud Secret Manager injetará variáveis de ambiente nos containers em tempo de execução, garantindo <i>Zero Trust</i> .

3. Especificação Técnica: O "Cérebro" da Calculadora (Backend)

3.1. Abordagem Orientada a Configuração (Config-Driven)

Codificar 1108 funções isoladas em Python seria um erro arquitetural crasso, tornando a manutenção impossível caso a usina mude um equipamento. O backend será **Orientado a Configuração**. Ele lerá o arquivo JSON mestre (`memorial_de_calculo_balanco.json`) na inicialização do container para compilar as expressões e gerar dinamicamente as lógicas de cálculo na classe `BalanceEngine`.

- **Precisão Obrigatória e Fuga do Ponto Flutuante:** É terminantemente proibido o uso do tipo primitivo `float` do Python. Devido a arredondamentos binários, 100 iterações em `float` destruiriam a paridade exigida. O uso do tipo estruturado `decimal.Decimal` é mandatório.
- **Loop de Convergência e Graceful Degradation:** O motor iterará sobre o Grafo comparando o `current_state` com o `previous_state`. O loop cessa quando a variação máxima (Δ) em qualquer um dos 1108 nós for menor que `0.0001`. Se o motor atingir 100 ciclos sem convergência, ele aplicará uma degradação graciosa: o cálculo é interrompido

para evitar *Out of Memory* (OOM), o estado atual é devolvido, e uma *flag* de `convergence_error: true` é enviada à interface.

3.2. Inteligência de Otimização: Goal Seek (Atingir Meta)

Para a funcionalidade de busca de metas, derivar analiticamente o grafo de 1108 nós é computacionalmente inviável. Portanto, o serviço utilizará métodos numéricos baseados em caixa-preta via `scipy.optimize`.

- A escolha técnica será o **Método da Secante** ou o **Algoritmo de Brent**. Eles fornecem a melhor aproximação de raízes sem exigir derivadas. O algoritmo envolverá o `BalanceEngine` instanciando execuções sucessivas, manipulando o *input* selecionado até que o *output* alvo atinja o valor especificado pelo usuário.

3.3. Inteligência de BI: Agregação Anual

Para o Planejamento Anual, carregar milhares de cálculos na memória do servidor web seria ineficiente.

- O backend fará o *offload* da carga pesada utilizando **Pandas**. Ele irá vetorizar as matrizes JSONB dos 12 cenários mensais aprovados e aplicar as lógicas descritas na tabela de `referencias_calculo`.
- Consequência arquitetural: Regras como "Média Ponderada" (ex: multiplicar o Brix pela Moagem do respectivo mês antes de tirar a média) serão executadas nativamente no DataFrame do Pandas em milissegundos, devolvendo apenas a matriz sintética consolidada para o React.

4. Frontend, Gerenciamento de Estado e UX

Construir formulários com 1108 campos exige um design de estado atômico rigoroso. Se um campo (ex: H3) acionar o re-render de todo o DOM, a aplicação ficará inutilizável.

- **Atomic Updates & Memoização Estrita:** A arquitetura do React exigirá o padrão de estado segregado (Slices). Ferramentas como **Zustand** (para estado global da barra de ferramentas) ou **React-Hook-Form** (para o registro não-controlado dos inputs) são mandatórias. Adicionalmente, todo componente de *Input/Output* deverá ser embrulhado em `React.memo()` com funções de comparação personalizadas (`areEqual`) para garantir que apenas o campo modificado pisque no navegador.
- **UX / Navegação Imersiva:** A interface será dividida fisicamente em **Abas por Setor** (Planejamento, Moagem, Caldo, Tratamento, Evaporação, Destilaria, Utilidades).
- **Barra de Contexto Padrão:** Um componente fixo no topo (`Sticky Header`) conterá o DNA temporal do cálculo: **Ano Safra, Mês, Versão** e o status do cenário. Esta barra abriga o botão **Calcular**.

- **Feedback de Processamento:** Como a requisição do motor recursivo pode levar alguns segundos (especialmente no *Goal Seek*), a arquitetura do UI deve implementar um padrão de bloqueio de tela suave (*Loading Overlay* ou *Skeleton Loaders*) para evitar duplos cliques e corrupção de estado de envio.

5. Arquitetura de Dados Relacional-Híbrida (Modelo Cloud SQL)

A persistência adota uma abordagem híbrida: a robustez transacional do PostgreSQL para as "gavetas" organizacionais e a flexibilidade NoSQL do JSONB para os dados dinâmicos. A indexação utilizará índices GIN no Postgres para permitir buscas dentro dos payloads JSONB no futuro.

Tabela	Coluna	Tipo	Descrição Arquitetural
cenarios	id	UUID	Chave primária universal (<code>uuid-oss</code>). Previne colisões e adivinhação de rotas.
cenarios	metadata	JSONB	Informações contextuais da simulação. Ex: <pre>{ "ano": 2026, "mes": "Abril", "versao": "v1", "status": "Aprovado" }</pre>
cenarios	valores_entrada	JSONB	Matriz exata dos IDs mapeados como "INPUT" no memorial original e seus valores preenchidos.
cenarios	valores_saida	JSONB	Matriz resultante dos cálculos (Outputs e Referências Circulares) gerada pelo motor Python.
cenarios	auth_id	String	Identificador do usuário provedor da ação. Garante a trilha de auditoria.
referencias	id_ref	String	Código padronizado do indicador. Ex: "H50" (Brix) ou "H1108" (Vapor).
referencias	regra_agregacao	Enum	Define como o Pandas tratará este ID no consolidado anual: <code>SOMA</code> , <code>MEDIA_SIMPLES</code> , <code>MEDIA_PONDERADA</code> .

6. Segurança e Integração Contínua (CI/CD)

A arquitetura foi projetada para um ambiente corporativo com rigorosos padrões de segurança, mitigando vazamentos de informações sensíveis da planta industrial.

- **Autenticação Zero-Trust:** O frontend não acessa o Cloud SQL diretamente sob nenhuma hipótese. O FastAPI fornecerá as rotas seguras validando o Token JWT injetado pelo Google SSO no cabeçalho `Authorization: Bearer <token>` em *todas* as requisições.
- **Isolamento de Rede:** O Cloud SQL (PostgreSQL) receberá um IP estritamente privado, residindo em uma VPC (Virtual Private Cloud). O Cloud Run se conectará a ele através do conector VPC *Serverless* (ou Cloud SQL Auth Proxy).
- **Gestão de Segredos:** Strings de conexão, chaves da API do Google, e senhas de banco nunca farão parte do repositório de código fonte (`.env` local apenas para desenvolvimento). O Google Secret Manager será o cofre oficial.
- **Pipeline de CI/CD Seguro:** O **Cloud Build** atuará como gatilho nos *commits* da branch `main`. A esteira seguirá os seguintes passos:
 1. *Linting* rigoroso do Python (`flake8`, `black`) e do React (`eslint`).
 2. Bateria de **Testes Unitários de Paridade** (Pytest testando amostras de JSON mestre contra os resultados conhecidos da planilha do cliente, com tolerância fixada em 4 casas decimais).
 3. Se e somente se os testes passarem, as imagens Docker serão *buildadas* e empurradas para o Artifact Registry.
 4. O Cloud Run instanciará a nova versão sem causar tempo de inatividade (*Zero Downtime Deployment*).